

PostgreSQL 16 — Safe Parameter Change Runbook for Large Restore on Streaming Replication

Environment

Item	Detail
PostgreSQL Version	16
Replication Type	Streaming Replication (with replication slot)
Data Directory	/apps/pgsql_data/16
Config File	/apps/pgsql_data/16/postgresql.conf

SECTION 1: ALTER SYSTEM vs. postgresql.conf — Best Practice

Recommendation: Use `ALTER SYSTEM` for temporary tuning changes

Aspect	ALTER SYSTEM	Direct postgresql.conf Edit
How it works	Writes to <code>postgresql.auto.conf</code> (overrides <code>postgresql.conf</code>)	Edits the main config file directly
Risk of human error	Low — PostgreSQL validates syntax	High — typos can prevent startup
Audit trail	<code>postgresql.auto.conf</code> shows only overrides	Hard to diff what changed
Rollback	<code>ALTER SYSTEM RESET parameter_name;</code> or <code>RESET ALL</code>	Must restore backup file manually
Precedence	<code>postgresql.auto.conf</code> values override <code>postgresql.conf</code>	Base config
Best for	Temporary tuning (like this restore scenario)	Permanent baseline config

Why ALTER SYSTEM wins for this scenario

- 1. Your original postgresql.conf stays untouched — no backup/restore of files needed
- 2. Rolling back is a single SQL command: ALTER SYSTEM RESET ALL;
- 3. You can verify what's overridden at any time:

```
bash
cat /apps/pgsql_data/16/postgresql.auto.conf
```

- 4. No risk of accidentally breaking the config file syntax
- 5. Works identically on both primary and standby

Important Rule

If a parameter exists in both postgresql.conf and postgresql.auto.conf, the value in postgresql.auto.conf ALWAYS wins. So after this procedure, always verify with SHOW parameter_name; inside psql.

SECTION 2: Safe Parameter Values for 400 GB Restore

Parameters to Change (with rationale)

Parameter	Typical Default	Restore-Optimized	Why
max_wal_size	1GB	10GB - 20GB	Reduces checkpoint frequency during heavy write load
min_wal_size	80MB	5GB	Keeps WAL files pre-allocated, avoids recycling too aggressively

checkpoint_timeout	5min	30min	Allows
longer intervals between			
			checkpoints
during restore			
+-----+-----+-----+-----+			
-----+			
checkpoint_completion	0.9	0.9	Keep at 0.9
(DO NOT lower this -			
_target			lower values
cause faster WAL			
			recycling!)
+-----+-----+-----+-----+			
-----+			
wal_buffers	-1 (auto ~64MB)	256MB	Larger
buffer for heavy WAL writes			
+-----+-----+-----+-----+			
-----+			
work_mem	4MB	256MB	Speeds up
sorts, joins during FK			
			creation
+-----+-----+-----+-----+			
-----+			
maintenance_work_mem	64MB	2GB	Speeds up
CREATE INDEX, FK			
			validation,
VACUUM			
+-----+-----+-----+-----+			
-----+			
wal_keep_size	0	50GB	CRITICAL:
Keeps WAL segments even			
			if standby
falls behind			
+-----+-----+-----+-----+			
-----+			
max_slot_wal_keep_size	-1 (unlimited)	-1 (unlimited)	CRITICAL:
Ensure this is -1 so			
			replication
slot is NOT overridden			
+-----+-----+-----+-----+			
-----+			
full_page_writes	on	on	NEVER turn
this off - data			

			corruption
risk			
+-----+	+-----+	+-----+	+-----+
-----+			
synchronous_commit	on	off (optional)	Can turn off
temporarily for speed;			
			small crash
risk window			
+-----+	+-----+	+-----+	+-----+
-----+			

CRITICAL Safety Parameters (these protect replication)

These two parameters are what **prevented your standby from surviving** last time:

1. `wal_keep_size = '50GB'` — This tells PostgreSQL: "Even after a checkpoint, keep at least 50 GB of old WAL segments on disk." This is your safety net if the standby temporarily disconnects.
2. `max_slot_wal_keep_size = '-1'` — This tells PostgreSQL: "Never forcibly remove WAL that a replication slot says is still needed." If this is set to a finite value (e.g., `10GB`), PostgreSQL will **invalidate the slot** and remove WAL when that limit is exceeded — which is exactly what happened to you.

SECTION 3: Step-by-Step Procedure

PHASE 1: Pre-Change Verification (Do this FIRST)

Run on **PRIMARY**:

```
sql
```

```
-- 1. Check current replication status
SELECT pid, username, client_addr, state, sent_lsn, write_lsn,
       flush_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;

-- 2. Check replication slot health
SELECT slot_name, slot_type, active, restart_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS slot_lag
FROM pg_replication_slots;

-- 3. Record current parameter values (screenshot or save this output)
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW checkpoint_completion_target;
SHOW wal_buffers;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
SHOW synchronous_commit;

-- 4. Verify slot is ACTIVE
SELECT slot_name, active FROM pg_replication_slots;
-- Must show: active = t
```

Run on **STANDBY**:

```
sql

-- Verify standby is in recovery and streaming
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

DO NOT PROCEED if replication is not healthy.

PHASE 2: Apply Parameter Changes

Order: STANDBY FIRST, then PRIMARY

Why standby first?

- The standby is the consumer of WAL. If you restart the primary first, you generate new WAL with no consumer running — widening the gap.
 - Restarting the standby first means it comes back up quickly and is ready to consume WAL the moment the primary restarts.
-

Step 2a: Apply on STANDBY

Connect to the standby via psql:

```
sql
-- Apply restore-optimized parameters
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
```

Some of these require a restart, some only a reload. Parameters requiring **restart**: `wal_buffers`

Parameters requiring only **reload**: `max_wal_size`, `min_wal_size`, `checkpoint_timeout`, `checkpoint_completion_target`, `work_mem`, `maintenance_work_mem`, `wal_keep_size`, `max_slot_wal_keep_size`, `synchronous_commit`

Since `wal_buffers` requires a restart, we must restart the standby:

```
bash
# Restart standby
sudo systemctl restart postgresql-16
# or
pg_ctl -D /apps/pgsql_data/16 restart
```

Wait and verify standby is streaming again:

```
sql
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Wait until: streaming
```

On PRIMARY, confirm the slot is active again:

```
sql

SELECT slot_name, active FROM pg_replication_slots;
-- Must show: active = t before proceeding
```

Step 2b: Apply on PRIMARY

Connect to the primary via psql:

```
sql

-- Apply restore-optimized parameters
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';

-- Optional: for extra speed (tiny crash-safety trade-off)
-- ALTER SYSTEM SET synchronous_commit = 'off';
```

Restart primary:

```
bash

sudo systemctl restart postgresql-16
# or
pg_ctl -D /apps/pgsql_data/16 restart
```

Immediately verify replication reconnects:

```
sql
```

```
-- On PRIMARY
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS slot_la
FROM pg_replication_slots;
-- active must be: t

SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must be: streaming
```

PHASE 3: Perform the 400 GB Restore

During the restore, monitor replication every 15-30 minutes:

```
sql
-- Run this on PRIMARY periodically
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS slot_la
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_replication_slots s
LEFT JOIN pg_stat_replication r ON s.slot_name = r.slot_name;

-- Check WAL disk usage
SELECT pg_size_pretty(sum(size)) AS total_wal_size
FROM pg_ls_waldir();

-- Check disk space (run in bash)
-- df -h /apps/pgsql_data/16/pg_wal
```

Warning signs to watch for:

- `active = f` on replication slot → standby disconnected, investigate immediately
- `slot_lag` growing beyond 30-40 GB → standby falling behind, but safe if `wal_keep_size` and slot are protecting WAL
- WAL directory consuming too much disk → you may need to pause the restore

PHASE 4: Post-Restore — Revert Parameters

After restore is complete, FK creation is done, and everything is verified:

Step 4a: Revert on STANDBY FIRST

```
sql

-- Option A: Reset ALL overrides at once (recommended)
ALTER SYSTEM RESET ALL;

-- Option B: Reset individual parameters
ALTER SYSTEM RESET max_wal_size;
ALTER SYSTEM RESET min_wal_size;
ALTER SYSTEM RESET checkpoint_timeout;
ALTER SYSTEM RESET checkpoint_completion_target;
ALTER SYSTEM RESET wal_buffers;
ALTER SYSTEM RESET work_mem;
ALTER SYSTEM RESET maintenance_work_mem;
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;
ALTER SYSTEM RESET synchronous_commit;
```

Restart standby:

```
bash

sudo systemctl restart postgresql-16
```

Verify standby is streaming:

```
sql

SELECT pg_is_in_recovery();
SELECT status FROM pg_stat_wal_receiver;
```

On PRIMARY, confirm slot is active:

```
sql

SELECT slot_name, active FROM pg_replication_slots;
```

Step 4b: Revert on PRIMARY

```
sql
```

```
ALTER SYSTEM RESET ALL;
```

Restart primary:

```
bash
```

```
sudo systemctl restart postgresql-16
```

Final verification:

```
sql
```

```
-- On PRIMARY: confirm all parameters are back to original
```

```
SHOW max_wal_size;
```

```
SHOW min_wal_size;
```

```
SHOW checkpoint_timeout;
```

```
SHOW wal_buffers;
```

```
SHOW work_mem;
```

```
SHOW maintenance_work_mem;
```

```
SHOW wal_keep_size;
```

```
SHOW max_slot_wal_keep_size;
```

```
-- Confirm replication is healthy
```

```
SELECT slot_name, active,  
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS slot_lag  
FROM pg_replication_slots;
```

```
SELECT client_addr, state,  
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag  
FROM pg_stat_replication;
```

Verification: Confirm postgresql.auto.conf is clean

```
bash
```

```
cat /apps/pgsql_data/16/postgresql.auto.conf
```

After `ALTER SYSTEM RESET ALL`, this file should be empty (or contain only a comment header). If it still has entries, something was missed.

SECTION 4: Restart Order Summary

+-----+-----+-----+		
----+		
Phase	First	Second
+-----+-----+-----+		
----+		
Applying changes	STANDBY first	PRIMARY second
(before restore)	(restart → verify	(restart → verify
	streaming)	replication
reconnects)		
+-----+-----+-----+		
----+		
Reverting changes	STANDBY first	PRIMARY second
(after restore)	(restart → verify	(restart → verify
	streaming)	replication
reconnects)		
+-----+-----+-----+		
----+		

The rule is always: STANDBY FIRST, PRIMARY SECOND.

Reason: The standby restarts faster (no recovery replay needed in most cases). By the time you restart the primary, the standby is already up and ready to consume WAL immediately. This minimizes the window where WAL accumulates with no consumer.

SECTION 5: Emergency — What If Replication Breaks During Restore

If the standby falls behind and you see the dreaded error:

```
ERROR: requested WAL segment has already been removed
```

Recovery Steps:

```
sql
```

```
-- On PRIMARY: drop the broken slot
SELECT pg_drop_replication_slot('your_slot_name');
```

bash

```
# On STANDBY: rebuild from scratch
sudo systemctl stop postgresql-16
rm -rf /apps/pgsql_data/16/*      # CAREFUL: only on standby!

pg_basebackup -h <primary_host> -p 5432 -U replication_user \
  -D /apps/pgsql_data/16 \
  -Fp -Xs -P -R \
  -S your_slot_name -C

sudo systemctl start postgresql-16
```

The `(-R)` flag auto-creates `standby.signal` and configures `primary_conninfo`. The `(-S slot_name -C)` creates a new replication slot during the basebackup.

SECTION 6: Quick Reference — ALTER SYSTEM Commands

Apply restore tuning:

```
sql

ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
```

Revert everything:

```
sql

ALTER SYSTEM RESET ALL;
```

Verify overrides are cleared:

```
bash
```

```
cat /apps/pgsql_data/16/postgresql.auto.conf
```

Key Takeaways

1. Always use `ALTER SYSTEM` for temporary parameter changes — your `postgresql.conf` stays untouched
2. Always restart **STANDBY** first, **PRIMARY** second — both when applying and reverting
3. Always set `wal_keep_size` and `max_slot_wal_keep_size = -1` when doing heavy write operations — these are your replication safety nets
4. Always verify replication is healthy after every restart before proceeding
5. Monitor slot lag during the restore — catching problems early prevents full standby rebuilds
6. `ALTER SYSTEM RESET ALL` is your one-command rollback — no file backups needed